

Zero-Code Robotics
LLMs, VLMs, and Prompt Engineering
for Mobile Robotics Applications

Pranav Shukla

Rachna Kadam

Ronish Thangadurai Nadar

Mechanical and Aerospace Engineering
NYU Tandon School of Engineering

Advisor: Prof. Vikram Kapila

Abstract

This project presents a zero-code framework that enables users to control mobile robots using natural-language instructions through a Python-based GUI. The system uses an overhead multi-camera setup for perception, where cameras are calibrated using a Charuco board and stitched into a global top-down view using homography alignment with ArUco markers. Robot localization is performed externally by detecting ArUco tags mounted on each sensor-less differential-drive robot, and dynamic obstacles are tracked using a background reference model with Gaussian blur and background subtraction.

High-level task reasoning is handled using Gemini 2.5 Pro, while Moondream is used for VLM-based object understanding (query/detect/point). Motion execution combines an RRT* planner with online replanning in dynamic scenes and a hybrid controller that uses Pure Pursuit for navigation and a PID terminal controller for precise pick-and-place alignment. Coordination across modules is supported through an RPC server (data, robot, and task services) and a flexible wireless stack (Wi-Fi, BLE, ESP-NOW) via an ESP32 bridge.

We validate the framework using three experiments: ordering numbers under a flipping constraint, spelling words with constrained placement direction, and collaborative two-robot color sorting with collision avoidance via replanning. The results show that LLM-driven task decomposition and scheduling can be reliably grounded in real-world robot execution when supported by robust perception, planning, and control.

Contents

Abstract	i
Contents	ii
List of Figures	v
1 Introduction	1
1.1 Motivation	1
1.2 Problem Formualtion	1
1.3 Constraints and Challenges	2
1.4 Objective and Scope of Work	2
2 Literature Review	4
2.1 Work in LLM-based Function Calling	4
2.2 LLM-based Navigation	4
2.3 LLM-based Planning	5
2.3.1 Communication Architectures in Multi-Robot Systems (MRS) . .	5
2.3.2 Recent LLM-MRS Integrations	5
2.4 LLMs for Task Scheduling and Execution	6
3 Novelty and Contributions	7
4 System Overview	8
4.1 Hardware Setup	8
5 Model	10
5.1 Overview	10
5.2 Perception	11
5.3 Localization	11
5.4 VLM Detection	11
5.5 Planner	12
5.5.1 Planning space and collision model	12

5.5.2	Safety clearance via obstacle inflation	12
5.5.3	Optimization objective	13
5.5.4	Path validity check and replanning trigger	13
5.6	Controller	13
5.6.1	Pure Pursuit path tracking	13
5.6.2	Terminal PID for precision docking	14
5.6.3	Switching logic and replanning compatibility	14
5.6.4	Gripper offset compensation (pick/place)	15
5.7	Communication	15
5.8	RPC System	15
5.9	Thinking LLM	16
6	Experiments and Results	17
6.1	User Interface (GUI)	17
6.2	Results	19
6.2.1	Ordering Numbers	19
6.2.2	Spelling (Word Formation: CAT and DOG)	20
6.2.3	Multi-Robot Color Sorting (Collaborative Sorting + Collision Avoidance)	21
7	Conclusion and Future Work	23
7.1	Conclusion	23
7.2	Future Scope	23
	Bibliography	25
A	Appendix	26
A.1	Hardware and Software Setup	26
A.1.1	Robot Platform (Differential Drive)	26
A.1.2	RPC Server Architecture	26
A.1.3	Graphical User Interface (GUI)	26
A.2	AI and Perception Stack	27
A.2.1	Large Language Models (LLM)	27
A.2.2	Vision-Language Model (VLM)	27
A.2.3	Planning and Navigation	27
A.3	Tool Registry	27
A.3.1	Display & Visualization	27
A.3.2	Line Following & Warping	27
A.3.3	Mathematical & Planning Utilities	28
A.3.4	Perception & Detection	28

A.3.5	Controller & Execution	28
A.3.6	Shape Generation (Pattern Formation)	28
A.3.7	System Management	28
A.4	Agent Architectures	29
A.4.1	Agent List	29
A.4.2	Example Agent Specification: Word formation agent	29

List of Figures

4.1	Hardware setup of the multi-robot system (overhead cameras, arena, obstacles, objects, and robots).	8
5.1	Model Architecture	10
6.1	Dashboard-style GUI for system interaction.	17
6.2	Ordering Numbers experiment: initial random placement (top row) and final ordered placement (bottom row).	19
6.3	Spelling experiment: forming the words CAT and DOG from randomly placed letter blocks.	20
6.4	Multi-robot color sorting experiment using two robots, including an example instance of online replanning for collision avoidance.	21

Chapter 1

Introduction

1.1 Motivation

The field of robotics and industrial automation is currently undergoing a paradigm shift, moving from rigid, code-heavy control schemes to flexible, conversational interfaces. Historically, deploying a robot for a domestic or industrial task required specialized knowledge of programming languages (C++, Python), kinematic models, and control theory. This barrier to entry has limited the adoption of automation in dynamic environments where tasks change frequently.

We are now transitioning to an era of Agentic Automation. In this new paradigm, the human operator no longer functions as a programmer defining every micro-step, but as a supervisor communicating high-level intent. The motivation behind this project is threefold: first, to automate complex domestic and industrial tasks using this agentic approach; the second to provide robust frameworks to deconstruct complex tasks into a series of feasible actions and lastly, to provide an educational platform that bridges the gap for industry engineers, helping them transition from traditional determinism to modern, AI-driven autonomy. By enabling a robot to understand natural language, we democratize access to robotics, allowing users to simply say, "check the inventory in the back," rather than writing a script to perform navigation and object detection.

1.2 Problem Formualtion

Large Language Models (LLMs) have demonstrated unprecedented capabilities in information retrieval, reasoning, and code generation. However, a fundamental question remains: Just how good are LLMs at physical agency?

While LLMs excel at generating text, the physical world requires precise, geometric actions. A model can easily describe how to pick up an object, but executing that action requires exact coordinate transformations, collision avoidance, and kinematic plan-

ning. The core problem this project addresses is the "Grounding Problem"—translating abstract linguistic tokens (e.g., "fast", "near", "carefully") into concrete numerical constraints (e.g., velocity limits, Cartesian coordinates, grip force). We must determine if an LLM can function not just as a chatbot, but as a reliable central processor for a robotic system.

1.3 Constraints and Challenges

Deploying LLMs on physical hardware introduces a unique set of constraints that are often absent in purely software-based AI tasks:

- **Computational Efficiency**

LLMs are resource-intensive. Running inference for every minor decision introduces latency that can be dangerous in real-time robotics. The system must balance the intelligence of large models with the speed required for motor control.

- **Problem Complexity**

Real-world environments are unstructured and dynamic. Unlike a chess game, the state space is infinite. The system must handle occlusions, lighting changes, and dynamic obstacles (handled via Background Subtraction and Gaussian Blur in this project) without requiring the LLM to process every pixel.

- **Self-Correction**

A critical limitation of current generative models is hallucination—confidently stating incorrect information. In a robotic context, a hallucinated path can lead to physical collision. Therefore, the system requires a robust feedback loop where the robot can verify the feasibility of an LLM-generated plan against physical reality (e.g., verifying a target location via VLM before moving) and correct itself if the action fails.

1.4 Objective and Scope of Work

The primary objective of this project is the development of a No-Code Agentic Framework that integrates high-level cognitive planning with low-level hardware execution. Scope of work includes:

1. **Development of a Task Planning Interface**

A software layer where users interact via natural language. This interface utilizes LLMs for task deconstruction and scheduling.

2. Seamless Hardware Integration

Bridging the gap between the "Thinking" layer (LLM) and the "Acting" layer (Robot). This involves integrating motion planning algorithms (RRT*, Pure Pursuit) and perception modules (Camera stitching, Aruco localization, VLM detection).

3. Environment Integration

Creating a robust communication pipeline (Wifi, ESP-Now, RPC System) that allows the robot to query its environment and receive feedback.

4. Educational deliverables

Providing sample applications and tutorials to demonstrate how traditional control loops (PID) can be hybridized with AI agents.

The result is a holistic system where a user interacts via a GUI, and the robot autonomously handles the perception, planning, and control required to execute the mandate.

Chapter 2

Literature Review

The integration of Large Language Models (LLMs) into robotics represents a significant convergence of Natural Language Processing (NLP) and control theory. This review categorizes recent advancements into four critical areas relevant to the proposed work: Function Calling, Navigation, Planning, and Task Scheduling.

2.1 Work in LLM-based Function Calling

The capability of LLMs to interface with external software tools—known as "tool use" or "function calling"—is foundational to the agentic approach. Early iterations of LLMs were isolated text processors, but works such as Toolformer (Schick et al., 2023) demonstrated that LLMs could teach themselves to use external APIs to solve problems they could not handle alone. This concept was further refined by Gorilla (Patil et al., 2023), which fine-tuned models specifically to generate accurate API calls, reducing hallucination in parameter generation.

In the context of robotics, function calling bridges the gap between natural language instructions and hardware control interfaces. HuggingGPT (Shen et al., 2024) proposed a controller framework where the LLM acts as a central dispatcher, selecting expert models (functions) to handle specific sub-tasks like object detection or pose estimation. This directly supports the proposed project's "no-code framework," where the LLM must dynamically select appropriate tools (e.g., `scan_environment()`, `activate_gripper()`) without explicit hard-coding by the user.

2.2 LLM-based Navigation

Traditional navigation relies on geometric maps and costmaps (SLAM), which lack semantic understanding. Recent literature focuses on "Grounding"—connecting language to physical space. LM-Nav (Shah et al., 2023) introduced a method to combine pre-

trained visual navigation models with LLMs, allowing robots to follow instructions like "go to the kitchen" without requiring labeled navigation data.

Further advancing this, VLM-Maps (Huang et al., 2023) and NLMMap-SayCan (Chen et al., 2023) utilize Visual Language Models (VLMs) to build semantically queryable maps. In these systems, the robot does not just see obstacles; it perceives "chairs," "doors," or "danger zones." This aligns with this project's use of Aruco markers and VLM detection for localization, where the agent must correlate a user's high-level query with low-level coordinate data for the Pure Pursuit controller.

2.3 LLM-based Planning

Planning is the cognitive bridge between a high-level goal and low-level actuation. While classical planners like A* or RRT* are geometrically optimal, they cannot reason about abstract tasks. SayCan (Ahn et al., 2022) was a landmark paper establishing that an LLM can act as a high-level planner if its outputs are "grounded" by value functions indicating what is physically possible (affordances).

However, LLMs often struggle with long-horizon reasoning. To address this, Tree of Thoughts (ToT) (Yao et al., 2023) and Reasoning via Planning (RAP) (Hao et al., 2023) introduced methods where the LLM explores multiple reasoning paths before committing to an action. For this project, which integrates RRT* for motion with an LLM for logic, the literature suggests a hierarchical approach: the LLM functions as the "semantic planner" (deciding what to do), while the RRT* algorithm functions as the "geometric planner" (deciding how to move without collision).

2.3.1 Communication Architectures in Multi-Robot Systems (MRS)

Standard frameworks like ROS 2 provide robust communication backbones. Tardioli et al. (2019) explored networked control systems, establishing baselines for latency and reliability. While effective for explicitly coded behaviors, these traditional architectures often lack the flexibility to handle the unstructured task definitions provided by LLMs.

2.3.2 Recent LLM-MRS Integrations

More recently, studies have attempted to bridge this gap.

- Mandi et al. (2023) explored LLMs for high-level multi-robot planning but relied heavily on static prompts.
- Chen et al. (2024) introduced frameworks for generating code for robot teams, though often limited by the context window or lack of real-time state feedback.

Despite these advances, existing literature has not yet fully addressed the seamless integration of a centralized LLM (like Gemini) with a lightweight, function-sharing RPC layer for real-time task deconstruction in collaborative settings.

2.4 LLMs for Task Scheduling and Execution

Moving beyond single tasks to complex, multi-step industrial automation requires robust scheduling and self-correction. AutoGPT (Significant Gravitas, 2023) popularized the recursive agent loop, where an LLM generates a task list, executes the first step, critiques the result, and re-plans.

In robotic contexts, LLM+P (Liu et al., 2023) proposes converting natural language into PDDL (Planning Domain Definition Language) to leverage the reliability of classical solvers for scheduling. Furthermore, multi-agent frameworks like MetaGPT (Hong et al., 2024) demonstrate that assigning specific roles (e.g., a "Planner Agent" vs. a "Reviewer Agent") significantly improves success rates. This supports the project's use of distinct agents for task deconstruction and execution monitoring to ensure computational efficiency.

Chapter 3

Novelty and Contributions

To address the gaps identified in the Related Work, this paper introduces the following novel contributions:

1. Unified RPC-LLM Architecture

We propose a framework that combines an RPC server for efficient data/function sharing with the reasoning power of Gemini, allowing for real-time translation of natural language into distributed robot commands.

2. Task Deconstruction Pipeline

We introduce a specific pipeline for decomposing abstract group tasks (e.g., "form a circle," "sort these blocks") into assigned roles for individual agents using LLM reasoning.

3. App-Based Interface

Unlike CLI-based research tools, we present a user-facing application that simplifies the operation of complex multi-robot systems.

Chapter 4

System Overview

4.1 Hardware Setup

The hardware setup consists of an overhead vision system, a physical arena, and multiple mobile robots with wireless communication. The arena contains the working region along with predefined static obstacles, while dynamic obstacles can be introduced during runtime for testing replanning and coordination.

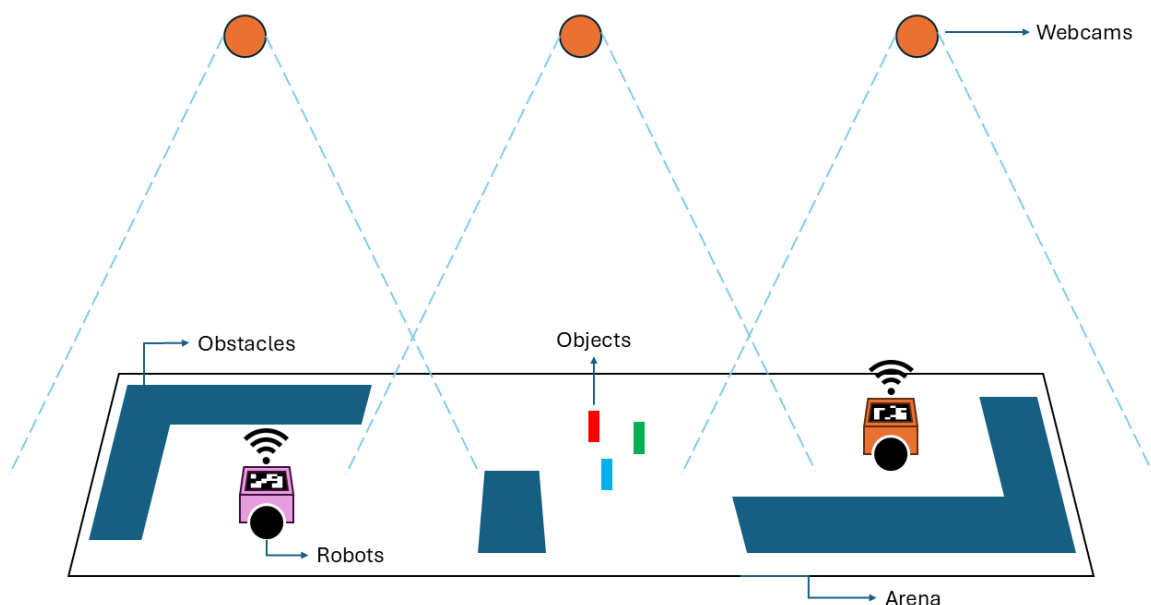


Figure 4.1: Hardware setup of the multi-robot system (overhead cameras, arena, obstacles, objects, and robots).

For perception, one or more webcams are mounted above the arena to provide a top-down view of the environment. These cameras are used for scene understanding, obstacle monitoring, and robot tracking. Each robot has an ArUco marker mounted on top, which enables reliable pose estimation from the overhead camera feed.

The robots are controlled through an ESP32-based communication interface. A central compute node runs the RPC server and planning/control stack, and periodically transmits motion and gripper commands to each robot. The receiver side on the robots is designed to support multiple wireless protocols (Wi-Fi, Bluetooth BLE, and ESP-NOW), allowing flexible communication depending on range and reliability requirements. The overall hardware layout is shown in Fig. 4.1.

Chapter 5

Model

5.1 Overview

The system is organized into interconnected modular layers that together enable perception, localization, planning, controller, communication, and high-level task reasoning for autonomous operation.

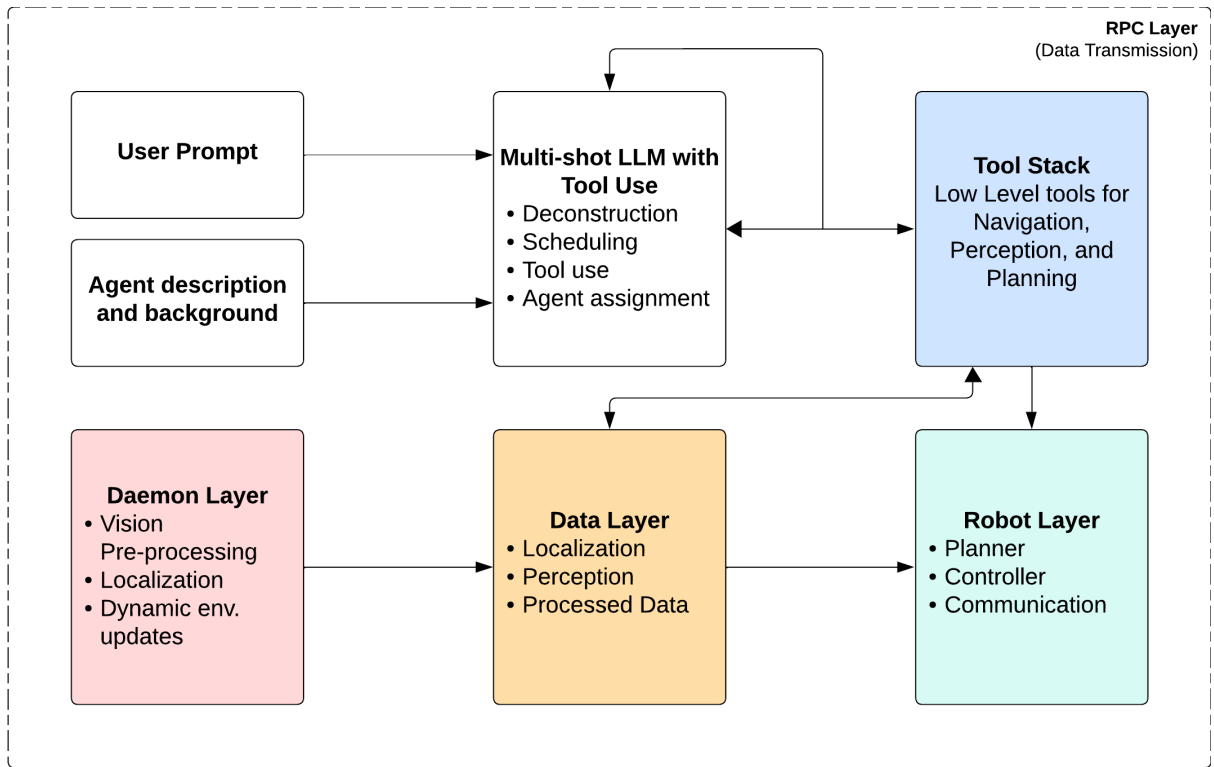


Figure 5.1: Model Architecture

The perception and localization modules process the visual camera input to understand the environment and estimate the pose of the multi-robots. These are further used by the planning module to generate feasible trajectories based on obstacles, which are then executed using control strategies for navigation and gripper manipulation. The commu-

nication module supports reliable data exchange across variety of robots. A higher level thinking task layer integrates language-based reasoning to decompose tasks, coordinate agents and schedule tasks. The overall model architecture is shown in Fig. 5.1.

5.2 Perception

Our system uses overhead-mounted webcams to perceive the environment and obtain a global top-down view of the arena. Each camera is initially calibrated using a Charuco board to correct lens distortion and ensure accurate geometric measurements.

After calibration, the homography matrices are computed for each camera by placing Aruco markers in the overlapping regions between the camera views. These markers allow us to estimate the relative position and orientation of every camera, enabling the frames to be warped and stitched into a single view of the workspace.

The stitched arena is then calibrated using the arena and static obstacles. This reference frame is used to detect dynamic obstacles during operation. Gaussian blur is applied to reduce noise, followed by background subtraction to track dynamic obstacles in the environment.

5.3 Localization

All the multi-robots are equipped with Aruco tags mounted on top, which is used to estimate the pose within the environment. The marker is detected in the stitched camera view, allowing the pose of the robots to be calculated in the global frame.

As the robots can appear in the field of view of multiple cameras simultaneously, multiple pose estimates of the same robot can be available at the same time. To improve localization accuracy, all such pose estimates are averaged to produce a more accurate estimate of the robot.

The estimated pose is then used to compute the pose of the gripper attached to the robot, enabling accurate control during manipulation tasks.

5.4 VLM Detection

The Vision-Language Model (VLM) is used in the system for high-level visual detection and scene understanding tasks. It enables the system to interpret the environment beyond geometric perception by reasoning about the objects present in the scene. The VLM is used for three main types of operations:

- The Query operation is used to identify and describe unknown objects in the environment based in visual input.

- The Detect operation is used to identify static obstacles, such as rectangular blocks and return their bounding boxes.
- The Point operation provides precise centroid coordinates of the detected objects, which are used for the gripper manipulation and interaction tasks.

5.5 Planner

For planning, we use RRT* to generate a collision-free path from the robot's current pose to the target. The planner operates on the current environment map, which includes static and dynamic obstacles, and other robots' positions in the environment.

The planner continuously checks the validity of the remaining path segment ahead of the robot. If any part of this path intersects with newly detected obstacles or violates a required safety clearance, the path is invalidated and replanning is initiated. The replanning is also triggered when the robot reaches the end of the current path segment but is still not sufficiently close to the final goal. To further increase robustness, periodic replanning is done at fixed intervals to account for gradual changes in the environment.

5.5.1 Planning space and collision model

We plan in 2D image/world coordinates with state $\mathbf{x} = [x \ y]^T \in [0, W) \times [0, H)$. Obstacles are represented as a union of polygons $\mathcal{O} = \bigcup_i \mathcal{O}_i$ (static, dynamic, and other robots treated as obstacles). A configuration is collision-free if it lies in the free space $\mathcal{X}_{free} = \mathcal{X} \setminus \mathcal{O}$.

5.5.2 Safety clearance via obstacle inflation

To enforce a required clearance, we inflate obstacles by a safety radius r_s :

$$\mathcal{O}^+ = \mathcal{O} \oplus \mathcal{B}(r_s),$$

where $\mathcal{B}(r_s)$ is a disk of radius r_s and $\oplus$ denotes the Minkowski sum. In practice, the environment is stored as an occupancy mask $M(y, x) \in \{0, 1\}$ and validity is checked by

$$\text{Valid}(\mathbf{x}) \Leftrightarrow M(\lfloor y \rfloor, \lfloor x \rfloor) = 0.$$

5.5.3 Optimization objective

Among feasible paths $\pi = \{\mathbf{x}_0, \dots, \mathbf{x}_K\}$ from $\mathbf{x}_s$ to $\mathbf{x}_g$, we select the shortest-length solution:

$$J(\pi) = \sum_{k=0}^{K-1} \|\mathbf{x}_{k+1} - \mathbf{x}_k\|_2, \quad \mathbf{x}_k \in \mathcal{X} \setminus \mathcal{O}^+.$$

5.5.4 Path validity check and replanning trigger

At runtime, we validate the remaining path segment by checking whether its waypoints (and/or connecting segments) remain in $\mathcal{X} \setminus \mathcal{O}^+$ under the updated map. If any part becomes invalid, the planner discards the path and replans from the current robot pose. Replanning is also triggered when the robot reaches the end of the current segment but remains outside the goal tolerance, and periodically at fixed intervals for robustness.

5.6 Controller

Our controller uses a hybrid approach:

- Pure Pursuit for fast navigation
- PID for precise final alignment

After the planner generates the path, the robot follows it using a Pure Pursuit controller. This works well for smooth and efficient motion, so it is used for most of the travel. However, Pure Pursuit is not accurate enough when the robot needs to do precision tasks like picking and placing with the gripper.

To handle this, once the robot enters a defined picking/dropping radius around the target, the controller switches to a PID-based controller. In this mode, the robot moves slower and makes small corrections in distance and heading to align properly. The final pose is also adjusted using the gripper offset, so the gripper reaches intended object location.

5.6.1 Pure Pursuit path tracking

Let the robot pose be (x, y, θ) and the current path be a polyline of waypoints $\{\mathbf{p}_i\}$. We select a *look-ahead point* $\mathbf{p}_L = (x_L, y_L)$ on the path such that

$$\|\mathbf{p}_L - [x \ y]^T\|_2 \geq L_d,$$

where L_d is the look-ahead distance (carrot point). The heading error to this point is

$$\alpha = \text{wrap}(\text{atan2}(y_L - y, x_L - x) - \theta),$$

and the Pure Pursuit curvature command is

$$\kappa = \frac{2 \sin(\alpha)}{L_d}, \quad \omega = v \kappa.$$

We then convert (v, ω) to differential wheel speeds using track width L :

$$v_R = v + \frac{\omega L}{2}, \quad v_L = v - \frac{\omega L}{2}.$$

To avoid driving forward when the target lies behind the robot, forward motion is suppressed for large $|\alpha|$ (turn-in-place behavior).

5.6.2 Terminal PID for precision docking

Near the goal, we switch to a slow “docking” controller based on distance and heading errors:

$$e_d = \sqrt{(x_g - x)^2 + (y_g - y)^2}, \quad e_\theta = \text{wrap}(\text{atan2}(y_g - y, x_g - x) - \theta).$$

Linear speed is reduced using a PID law on distance:

$$v = K_{p,d} e_d + K_{i,d} \int e_d dt + K_{d,d} \frac{de_d}{dt},$$

where $\frac{de_d}{dt}$ is computed numerically (e.g., $\frac{e_d(k) - e_d(k-1)}{\Delta t}$) and can be filtered to reduce noise. Angular speed is proportional to heading error:

$$\omega = K_{p,\theta} e_\theta.$$

The commanded wheel speeds are again $v_{R/L} = v \pm \omega L/2$, with saturation to respect actuator limits.

5.6.3 Switching logic and replanning compatibility

Let R_g be the switching radius. The controller runs Pure Pursuit while the robot is far from the final goal, then switches to terminal PID when

$$e_d \leq R_g \quad \text{or when the path is exhausted (end of waypoints).}$$

This makes the behavior robust: Pure Pursuit handles long-range motion efficiently, while PID improves final pose accuracy.

5.6.4 Gripper offset compensation (pick/place)

For manipulation, the robot center must stop behind the object so the gripper aligns with the target. If the desired gripper point is (x_o, y_o) and the approach heading is θ_a , we offset the robot center by d_g :

$$x_g = x_o - d_g \cos \theta_a, \quad y_g = y_o - d_g \sin \theta_a,$$

so that when the robot reaches (x_g, y_g) the gripper tip coincides with (x_o, y_o) .

5.7 Communication

The communication framework is designed to support multiple wireless protocols to ensure flexibility and reliability. It consists of three main components: Wi-Fi, Bluetooth (BLE), and ESP-NOW.

An ESP32 acts as a central transmitter and communicates serially with the RPC system server. Based on the controller outputs, the server periodically sends the appropriate control commands to each robot.

Each robot functions as a receiver, and the system is designed to be protocol-agnostic. Robots can receive commands over different protocols, allowing different communication modes to be used based on the range, latency and reliability requirements.

5.8 RPC System

To manage data flow and coordination across modules, we built an RPC-based data control system. In this setup, an RPC server runs centrally and hosts a set of services, while multiple RPC clients connect to it to read and write shared data.

The RPC system can be organized into three service layers:

1. Data Layer

The Data service stores global, shared state that is common to the entire system (all robots + environment). This includes the stitched camera frame, detected robot poses, static obstacles, dynamic obstacles, reference frame, and visualization data (like planned paths). The key part is that it's thread-safe, so perception can update the state while other modules read it without conflicts.

2. Robot Layer

The Robot service handles robot-specific operations, mainly command and path management. It maintains:

- a command buffer per robot (left speed, right speed, gripper action)

- a path list per robot (targets/waypoints)
- a periodic loop that continuously sends the latest commands to the hardware via serial (so the robots keep receiving updated control commands at a fixed rate)

3. Task Layer

The Task service exposes higher-level function calls that perform longer operations that don't belong in the low-level loop. For example, calibration runs as a task: it grabs the latest stitched frame, stores it as the reference frame, uses the VLM to detect static obstacles, and then saves those obstacles back into the Data service.

5.9 Thinking LLM

The “Thinking LLM” is the high level reasoning layer that converts a user's natural-language request into tool calls. It follows an agentic workflow: define an agent, give it the right tools, decompose the task, then execute schedule of actions. They are:

1. Agents

Users can create custom agents by specifying the agent's behavior, constraints, and operation style. This description, along with the list of tools allowed by the agents to use, forms the system prompt for that agent. The user's chat request is then provided as the user prompt.

2. Tools

Tools are low-level functions the LLM can call to interact with the system and environment, such as accessing the Data/Robot/Task layer functions via RPC.

3. Task deconstruction

Given the agent's prompt and the user request, the LLM breaks down the overall goal into smaller sub-tasks that map directly to the available tool calls.

4. Scheduling and execution

After deconstructing, the LLM forms an ordered task schedule and executes it by calling the selected tools in sequence. As each tool returns information, the LLM can update the plan and continue until the full user task is completed.

Chapter 6

Experiments and Results

6.1 User Interface (GUI)

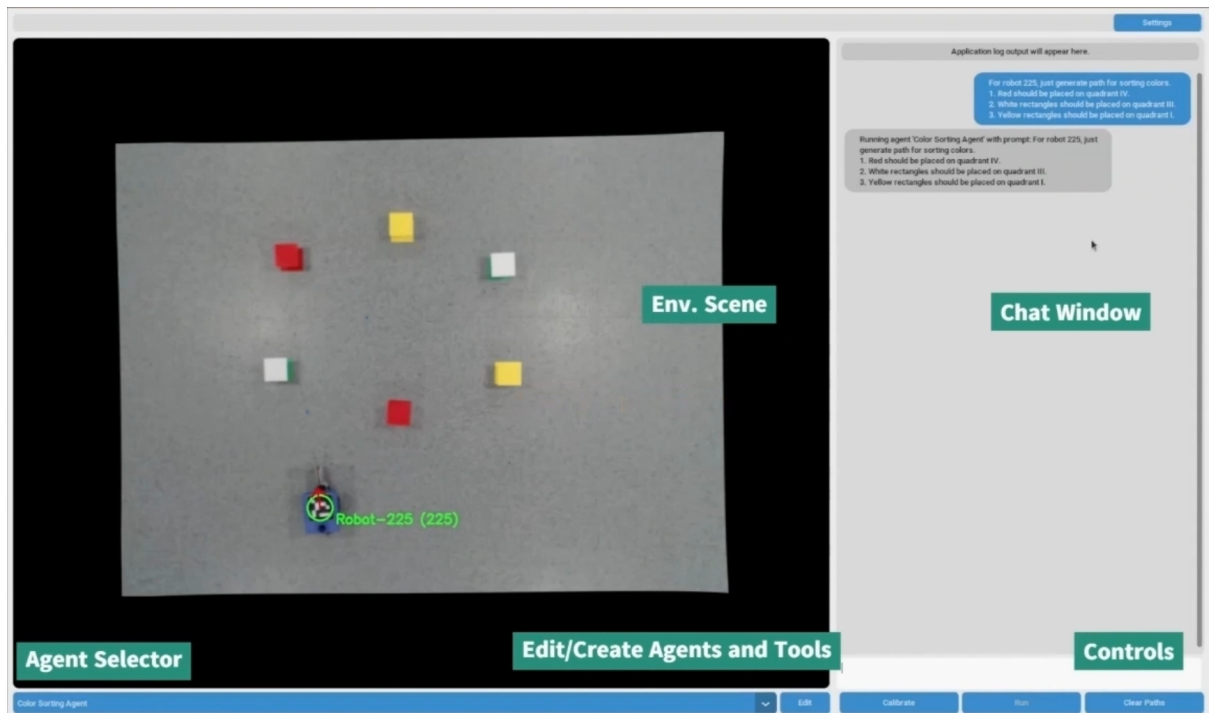


Figure 6.1: Dashboard-style GUI for system interaction.

To demonstrate how a user interacts with the overall system, we developed a dashboard-style GUI that combines the live environment view with a chat-based task interface. The GUI is designed to make the system easy to operate for non-technical users while still allowing control over agent behavior and system execution.

As shown in Fig. 6.1, the left side of the interface displays the global stitched frame obtained from all overhead cameras. This gives the user a real-time view of the arena, robot positions, and obstacles in a single unified scene. On the right side, the GUI

provides a chat window where the user can enter task requests in natural language (e.g., pick-and-place commands or navigation goals).

The bottom-left panel allows users to select and configure agents, including setting agent behavior and choosing which tools are available for execution. The bottom-right panel contains control buttons for core system actions: *Calibrate* (to initialize the scene and obstacles), *Run* (to execute tasks), and *Clear Paths* (to reset planned trajectories and visual overlays).

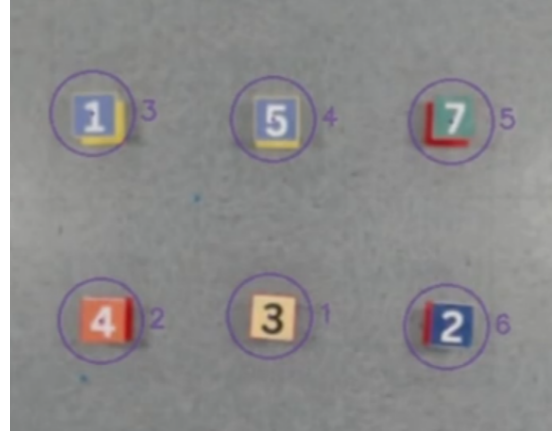
Overall, this interface connects high-level user intent with the underlying perception, planning, and multi-robot control pipeline in a single unified dashboard.

6.2 Results

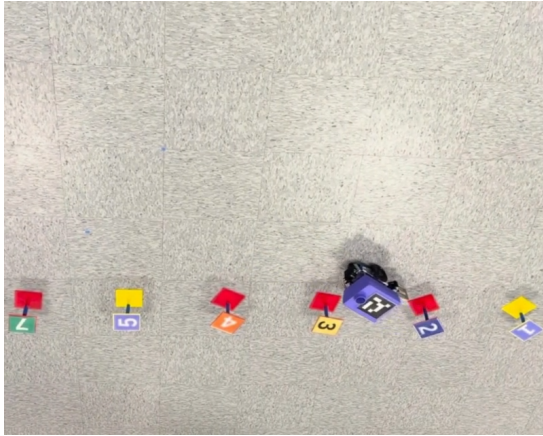
6.2.1 Ordering Numbers



(a) Before (view 1).



(b) Before (view 2).



(c) After (view 1).



(d) After (view 2).

Figure 6.2: Ordering Numbers experiment: initial random placement (top row) and final ordered placement (bottom row).

In this experiment, we evaluated the system’s ability to perform a multi-step pick-and-place task using VLM-based perception and LLM-based reasoning. The goal was to identify numbered blocks (1–7) randomly placed in the workspace and arrange them in ascending order.

A key constraint in this setup is that the numbers appear flipped after a block is dropped due to the gripper orientation. To handle this, the user instructed the system to begin placement from the **right side** of the target row and then continue placing blocks toward the **left**, ensuring the final arrangement reads correctly after flipping.

Fig. 6.2a and Fig. 6.2b show the initial configuration of the scene before execution, where the numbered blocks are placed randomly. Fig. 6.2c and Fig. 6.2d show the final outcome after execution. The task was completed successfully: the VLM detected all

numbered objects (1–7) and the system planned and executed the pick-and-place sequence to achieve the required ordering under the flipping constraint.

A demonstration video of this experiment is provided here: [Ordering Numbers Demo Video](#).

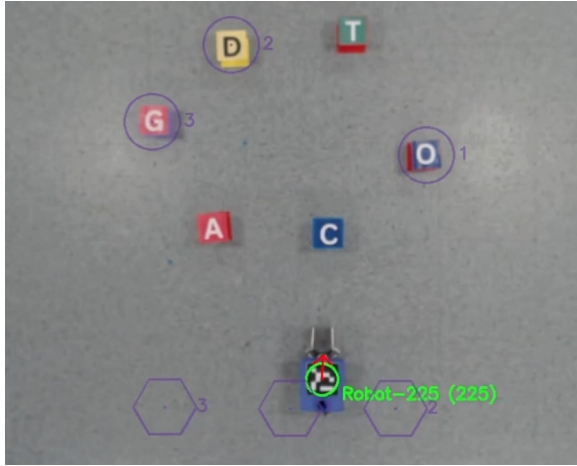
6.2.2 Spelling (Word Formation: CAT and DOG)



(a) CAT: Before.



(b) CAT: After.



(c) DOG: Before.



(d) DOG: After.

Figure 6.3: Spelling experiment: forming the words CAT and DOG from randomly placed letter blocks.

In this experiment, we tested the system on a word-formation task using letter blocks. The robot was asked to form the word **CAT** first, and then form **DOG** as a separate task. The letters were initially placed randomly in the workspace.

Similar to the ordering experiment, the letters appear flipped after dropping due to the gripper orientation. To ensure the final word reads correctly after placement, the user instructed the system to begin placing letters from the **right side** of the target region and then place subsequent letters toward the **left**.

Fig. 6.3a and Fig. 6.3b show the scene before and after forming **CAT**. Fig. 6.3c and Fig. 6.3d show the scene before and after forming **DOG**. The task was completed successfully: the VLM detected the randomly placed letters, verified that sufficient letters were available to form the requested word, and the system executed the required pick-and-place sequence to correctly spell the word under the flipping constraint.

A demonstration video of this experiment is provided here: [Spelling Demo Video](#).

6.2.3 Multi-Robot Color Sorting (Collaborative Sorting + Collision Avoidance)

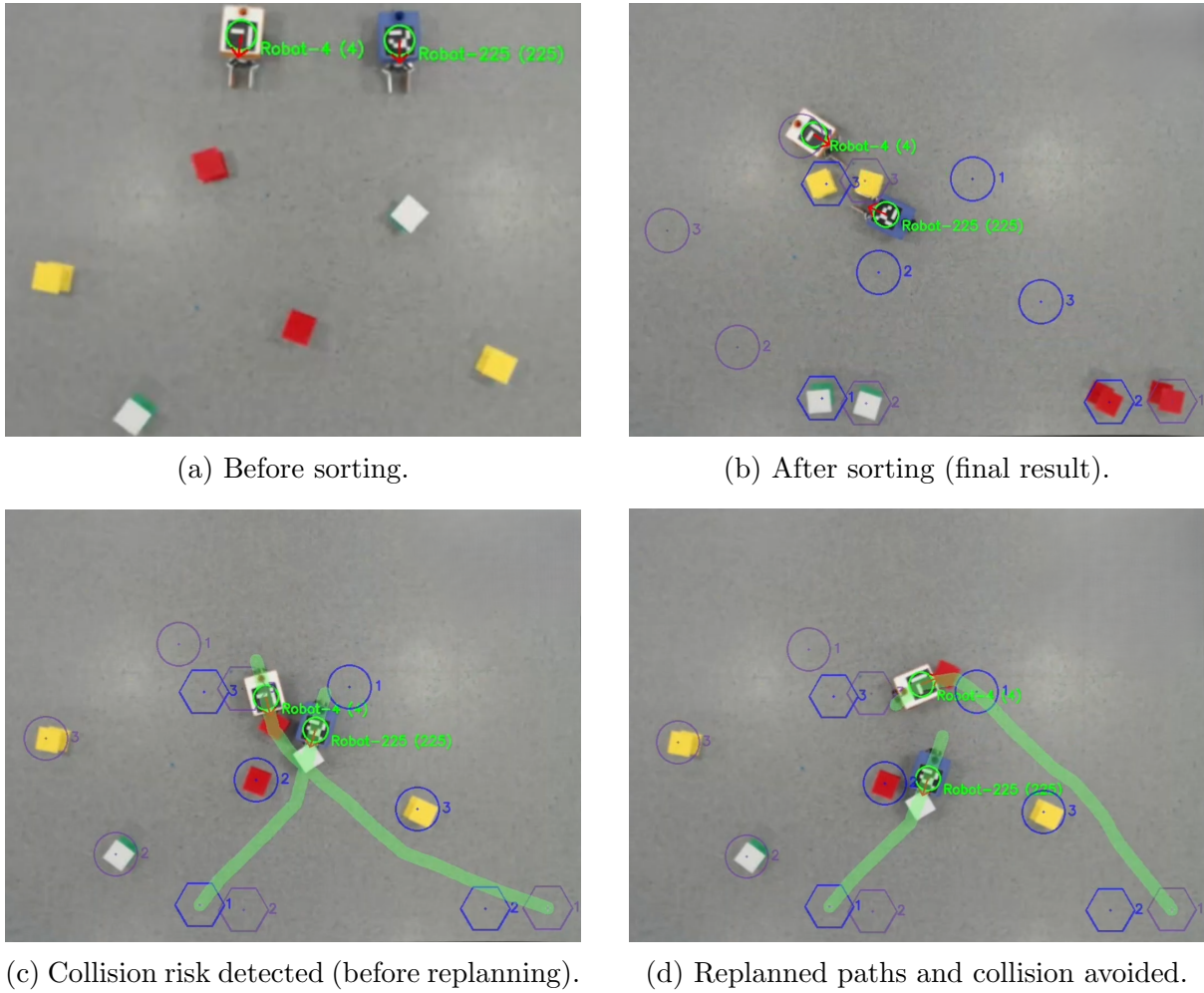


Figure 6.4: Multi-robot color sorting experiment using two robots, including an example instance of online replanning for collision avoidance.

In this experiment, we evaluated collaborative multi-robot task execution in a shared workspace. The system was given a high-level instruction to identify colored blocks in the scene and sort them into different quadrants, with **two robots working together** to complete the task.

The task was completed successfully. The system detected the colored blocks and executed coordinated pick-and-place actions to achieve the desired sorting: **yellow blocks were placed in the top-left quadrant, white blocks were placed in the bottom-right quadrant, and red blocks were placed in the bottom-right quadrant.** Fig. 6.4a shows the initial scene configuration, and Fig. 6.4b shows the final sorted outcome.

Since both robots operated simultaneously in the same environment, their planned paths occasionally brought them close to each other. During execution, the robots were observed to be on a potential collision course; however, the system detected this risk and triggered replanning before contact. The robots successfully updated their trajectories and avoided collision while continuing the sorting task. Fig. 6.4c and Fig. 6.4d highlight an example instance of this collision avoidance behavior.

A demonstration video of this experiment is provided here: [Multi-Robot Color-Sorting Demo Video](#).

Chapter 7

Conclusion and Future Work

7.1 Conclusion

In this work, we presented a comprehensive framework for multi-robot coordination driven by Large Language Models. By integrating Google’s Gemini with a robust RPC server architecture and robust controls, we successfully bridged the gap between high-level user intent and low-level robot execution.

Our system demonstrated the ability to:

1. Deconstruct Tasks

Effectively break down abstract commands into executable sub-tasks for shape formation and sorting.

2. Share Data Efficiently

Utilize an RPC server to manage function calls and data states

3. Simplify Interaction

Provide an intuitive app interface that lowers the barrier to entry for operating multi-robot teams.

The successful execution of these tasks confirms that general-purpose LLMs can serve as effective high-level planners for physical agents when supported by a strong controls backbone.

7.2 Future Scope

While the current system establishes a strong foundation, several avenues remain for future exploration and optimization:

- Integration of Symbolic Planners

To enhance reliability and safety, we plan to integrate symbolic planners (e.g., PDDL). This will allow the system to verify the logical consistency of LLM-generated plans before execution, reducing hallucinations and physical errors.

- Conversational Interface (Chat)

We intend to evolve the user interface from a single-input command system to a multi-turn chat interface. This will enable iterative refinement of tasks, and enable human intervention, allowing users to provide feedback or corrections while the robots are operating.

- Hierarchical Planning

Future iterations will implement hierarchical planning structures. By separating high-level strategy (assigned to cloud-based LLMs) from low-level control (handled by local controllers), we can improve reaction times and system robustness.

- Optimizing Local LLMs (SLMs)

To reduce latency and dependence on internet connectivity, we plan to experiment with Small Language Models (SLMs) deployed directly on the robot hardware (edge computing). These local models will handle immediate reasoning and minor adjustments, reserving the larger cloud models for complex, high-level orchestration.

- Complex Collaborative Tasks

We aim to scale the system to handle more intricate scenarios, such as collaborative transport of heavy objects or dynamic environmental exploration, which require tighter synchronization and continuous feedback loops.

Bibliography

- [1] Ahn, M., Brohan, A., Brown, N., et al., *Do as I can, not as I say: Grounding language in robotic affordances*, arXiv:2204.01691, 2022.
- [2] Chen, B., Xia, F., Ichter, B., Rao, K., & Gopalakrishnan, K., *Open-vocabulary queryable scene representations for real world planning*, arXiv:2209.09874, 2022.
- [3] Hao, S., Gu, Y., Ma, H., et al., *Reasoning with Language Model is Planning with World Model*, arXiv:2305.14992, 2023.
- [4] Hong, S., Zheng, X., Chen, J., et al., *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*, arXiv:2308.00352, 2023.
- [5] Huang, C., Mees, O., Zeng, A., & Burgard, W., *Visual Language Maps for Robot Navigation*, Proc. IEEE ICRA, 2023.
- [6] Liu, B., Jiang, Y., Zhang, X., et al., *LLM+P: Empowering Large Language Models with Optimal Planning Proficiency*, arXiv:2304.11477, 2023.
- [7] Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E., *Gorilla: Large Language Model Connected with Massive APIs*, arXiv:2305.15334, 2023.
- [8] Schick, T., Dwivedi-Yu, J., Dessì, R., et al., *Toolformer: Language models can teach themselves to use tools*, NeurIPS, 2023.
- [9] Shah, D., Osinski, B., Ichter, B., & Levine, S., *LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action*, CoRL, 2023.
- [10] Shen, Y., Song, K., Tan, X., et al., *HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face*, NeurIPS, 2023.
- [11] Significant Gravitas., *AutoGPT: An experimental open-source attempt to make GPT-4 fully autonomous*, GitHub Repository, 2023.
- [12] Yao, S., Yu, D., Zhao, J., et al., *Tree of thoughts: Deliberate problem solving with large language models*, arXiv:2305.10601, 2023.

Appendix A

Appendix

A.1 Hardware and Software Setup

A.1.1 Robot Platform (Differential Drive)

- **Chassis:** Custom “Bo-bots” with differential-drive kinematics and a gripper.
- **Computation:** Raspberry Pi 3 equipped with a Servo Hat.
- **Actuation:**
 - **Drive:** Parallax continuous-rotation servos.
 - **Manipulation:** Parallax standard servo for custom grippers.
- **Communication:** ESP32 modules utilizing a JSON-serialized interface with heart-beat mechanisms for connection stability.

A.1.2 RPC Server Architecture

To facilitate low-latency control, we utilize a custom RPC server built on ZeroMQ (ZMQ) and CBOR. The server employs a pool of four parallel workers to handle incoming requests from the LLM and the GUI.

A.1.3 Graphical User Interface (GUI)

The control interface is developed in Python using Tkinter, providing a lightweight dashboard for monitoring robot states and issuing manual overrides.

A.2 AI and Perception Stack

A.2.1 Large Language Models (LLM)

We evaluated several LLMs during development and testing:

- Llama-3-8B-Instruct
- Gemini-1.5-Flash
- Gemini-2.5-Pro

We observed that as models became larger and more capable, the need for custom architecture support became less critical.

A.2.2 Vision-Language Model (VLM)

To balance computational cost and response latency, we deployed a local VLM server using **Moondream**. This enables faster responses compared to querying large cloud-based VLMs for frequent visual checks.

A.2.3 Planning and Navigation

- **Path Planning:** We utilize RRT* (with support for fast replanning) for global path generation.
- **Obstacle Detection:** Dynamic obstacles are tracked using background subtraction combined with Gaussian blur and segmentation techniques using top-down camera feeds.

A.3 Tool Registry

The following functions are exposed to the LLM agents to execute physical tasks.

A.3.1 Display & Visualization

- `display_frame`: Displays an OpenCV image from a file in a window.

A.3.2 Line Following & Warping

- `detect_center_line_and_save`: Detects an orange center line in a warped image, samples points, and visualizes the trace.
- `warp_points / unwarp_points`: Transforms points between the original image space and the warped arena space.

A.3.3 Mathematical & Planning Utilities

- `add`: Basic addition calculator.
- `trace_targets`: Plans minimal waypoints for a robot to reach target coordinates using A* pathfinding.
- `pick_and_drop`: Plans a pick-and-drop path including approach and exit waypoints.

A.3.4 Perception & Detection

- `detect_robots`: Retrieves current poses (x, y, θ) of all robots.
- `detect_objects`: Detects objects based on text prompts (returns centroids).
- `find_objects`: Queries the vision model to list items present (semantic search).
- `point_objects` / `point_objects_list`: Returns centroids for single or multiple text prompts.

A.3.5 Controller & Execution

- `exec_robot_create_thread`: Spawns a background PID controller thread for a robot.
- `stop_robot_thread`: Halts a specific robot's controller.
- `create_code`: Writes generated Python code to a file.

A.3.6 Shape Generation (Pattern Formation)

- **Geometric Primitives**: `generate_circle_pattern`, `generate_rectangle_pattern`, `generate_trapezium_pattern`, `generate_parallellogram_pattern`, `generate_diamond_pattern`, `generate_ellipse_pattern`.
- **Coverage Paths**: `generate_lawnmower_pattern`, `generate_zigzag_pattern`, `generate_spiral`

A.3.7 System Management

- `rename_robot`: Updates/adds robot names.
- `get_robot_name` / `get_robot_id`: Resolves robot identifiers.
- `point_selection`: Launches a GUI for interactive target selection.

A.4 Agent Architectures

The system employs specialized agents, each equipped with a specific subset of the tools listed above.

A.4.1 Agent List

- Color Sorting Agent
- Conversational Agent
- Line Following Agent
- Number Sorting Agent
- Pattern Generation Agent
- Pick and Drop Agent
- Planning Agent
- Point Tracking Agent
- Programming Agent
- Super Market Planning Agent
- Word Formation Agent

A.4.2 Example Agent Specification: Word formation agent

- **Description:**

An agent that detects all letters in a scene and generates a robot pick-and-place plan to form a given target word. The robot must always pick the nearest required letter (Euclidean distance), updating its position after each pick. Letters are placed on a line below the scene with 150 px spacing. Because each placed letter is flipped by 180°, the agent must reverse the target word and place letters in that reversed order so the final readable word appears correctly.

- **System Instructions (System Prompt):**

You are a Word Formation Agent.

1. Inputs You Will Receive

You will be given:

A scene containing multiple alphabet characters in any order and at any locations.

The robots current position in pixel coordinates.

The target word that must be formed (example: "DOG").

Arena size: width = 980 px, height = 740 px.

2. Your Responsibilities

A. Detect Letters

Identify all alphabet letters present in the scene.

For each detected letter, return:

The character (A-Z)

Its position in the scene (x, y in pixels)

B. Pick Letters in Correct Sequence

The robot must always pick the nearest available letter first based on Euclidean distance from its current position.

Update the robot position after each pick (robot moves to the letters position).

3. Placement Rules

All letters must be placed below the current scene (i.e., arrangement line is below the pickup area).

The robot places letters with horizontal spacing of 150 px between consecutive letters.

The first placed letter defines the starting x,y position, spacing is applied relative to that.

4. IMPORTANT: Orientation Reversal

When the robot picks a letter and places it below, its orientation flips by 180 degrees.

Because of this:

If the target word is "DOG" (D-O-G when read normally),

The robot must place the letters in reverse order so that after flipping, the final readable word is in the correct forward order.

Example:

For target "DOG", the robot must place the letters in the order:

G -> O -> D

Because after the 180 rotation, the final readable result becomes:

D -> 0 -> G

You must always compute this reversed placement sequence automatically for any given target word.

- **User request (User Prompt):**

For robot 225, generate path to form the word CAT.